Funções de crescimento

Aula 03 - Estrutura de dados

Professor Daniel Marques

UEPB
Universidade
Estadual da Paraíba

Introdução

Na aula passada aprendemos a definir uma função que descreve o custo de execução de algoritmos

$$1.1 * n^2 + (10 + \sin(n + 15) * n^{1.5}) + 9000$$

Vimos exemplos simples cujas funções são também simples. Mas, observe o seguinte caso acima, a função que descreve o tempo de execução de um algoritmo qualquer.

Introdução

A Solução. Simplificar. Na verdade, nós já utilizamos algumas simplificações para facilitar a análise de algoritmos

$$1.1 * n^2 + (10 + \sin(n + 15) * n^{1.5}) + 9000$$

A principal delas é ignorar o custo real das operações primitivas, utilizando a constante C ou 1. Agora, faremos mais uma abstração simplificadora, chamada **ordem de crescimento**

Introdução

A ideia é determinar como o algoritmo se comporta para valores muito grandes de entrada. Neste caso, ignoramos as constantes e os valores de menor magnitude por entender que eles não são significativos diante dos valores de maior magnitude

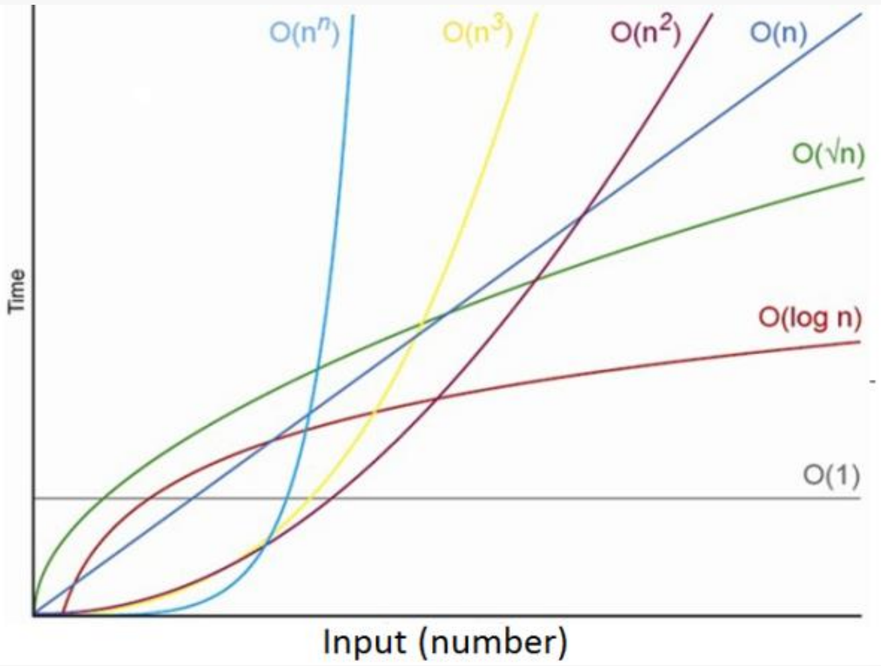
$$f(n) = \cancel{1.1} * n ** 2 \cancel{+ (10 + \sin(n+15)*n**1.5) + 9000}$$

Na prática, isso significa dizer que podemos:

- ignorar as constantes;
- ignorar os expoentes de menor magnitude.

Introdução

Agora sim, é muito mais direto olhar para n^2 e ter uma ideia clara do crescimento do tempo de execução do algoritmo, porque n^2 é uma função que já conhecemos bastante



Por exemplo, na escolha entre uma solução $\Theta(\log n)$ e uma solução $\Theta(n)$, é preferível a primeira, pois à medida que o tamanho da entrada cresce, o tempo de execução cresce mais lentamente que $\Theta(n)$

Introdução

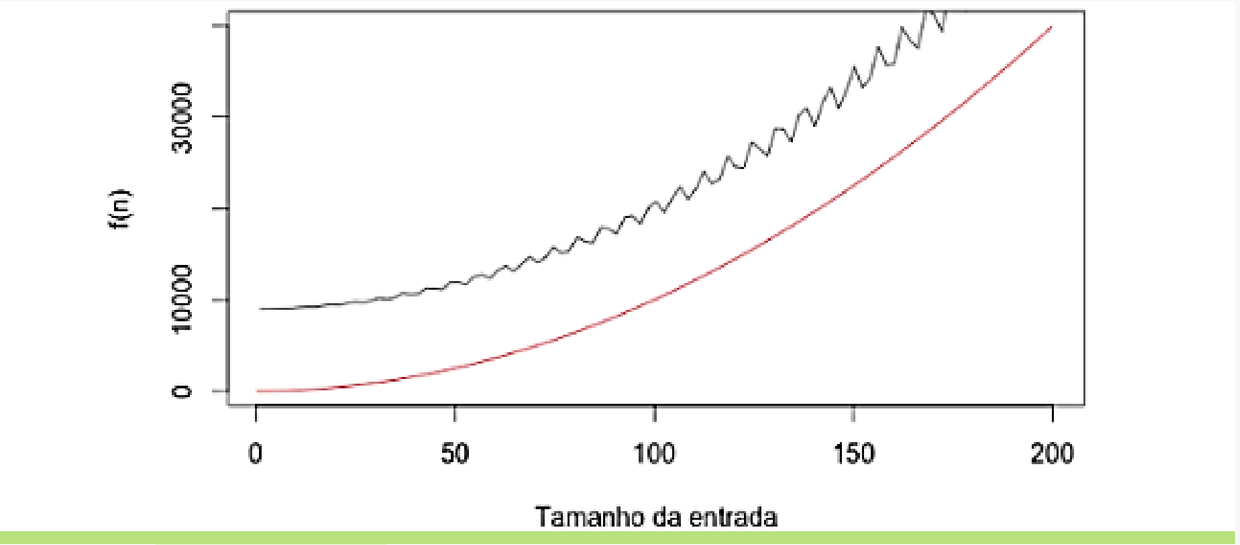
Não. Eu estou dizendo que elas pertencem à mesma classe de funções, as funções quadráticas

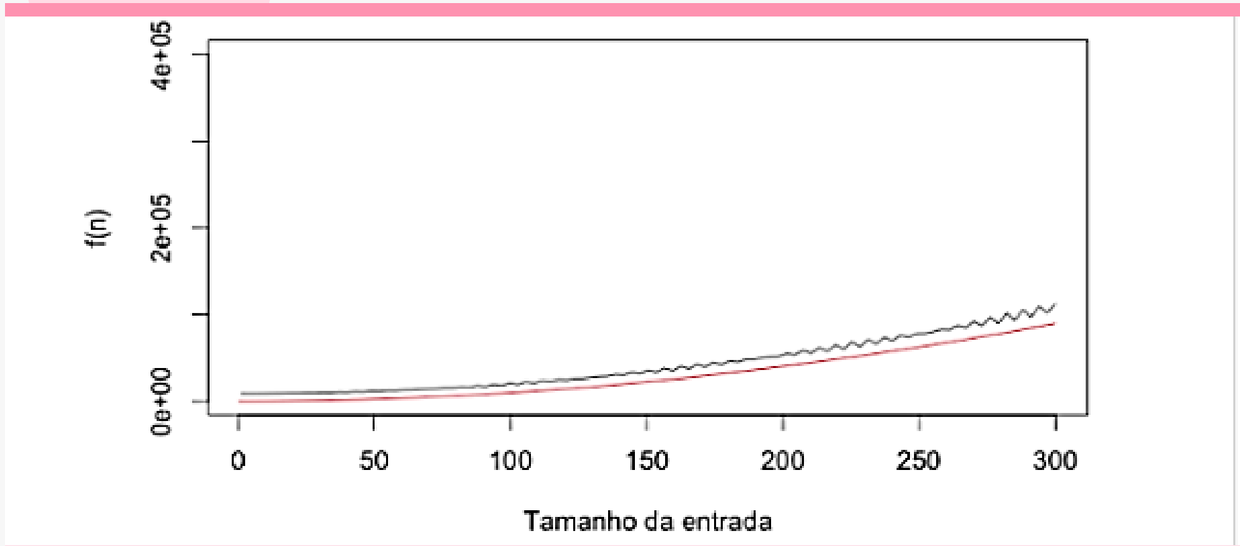
"Ainda não entendi. Você está querendo dizer que $f(n) = 1.1 * n^2 + (10 + \sin(n + 15) * n^{1.5}) + 9000$ e $g(n) = n^2$ são iguais?"

Eu estou querendo dizer que essas duas funções possuem a mesma ordem de crescimento para grandes entradas e que se aproximam muito uma da outra para grandes valores de n^2

Introdução

Vou te mostrar. Os gráficos da sequência abaixo ilustra essas duas funções. $f(n)$ está destacada em azul e $g(n)$ em vermelho





A única diferença é que a entrada (eixo x) vai aumentando de um gráfico para outro

Introdução

Através da identificação das operações primitivas e da quantidade de vezes que são executadas no algoritmo de busca linear chegamos à seguinte função da figura à direita

$$f(n) = 3 * c * n + 3 * c$$

Aplicando as diretrizes de simplificação, temos:

f(n) = ~~3 * c *~~ n ~~+ 3 * c~~

Isto é, $f(n) = 3 * c * n + 3 * c$ é $\Theta(n)$. Isso significa dizer que $f(n)$ tem a mesma ordem de crescimento que uma função linear.

Introdução

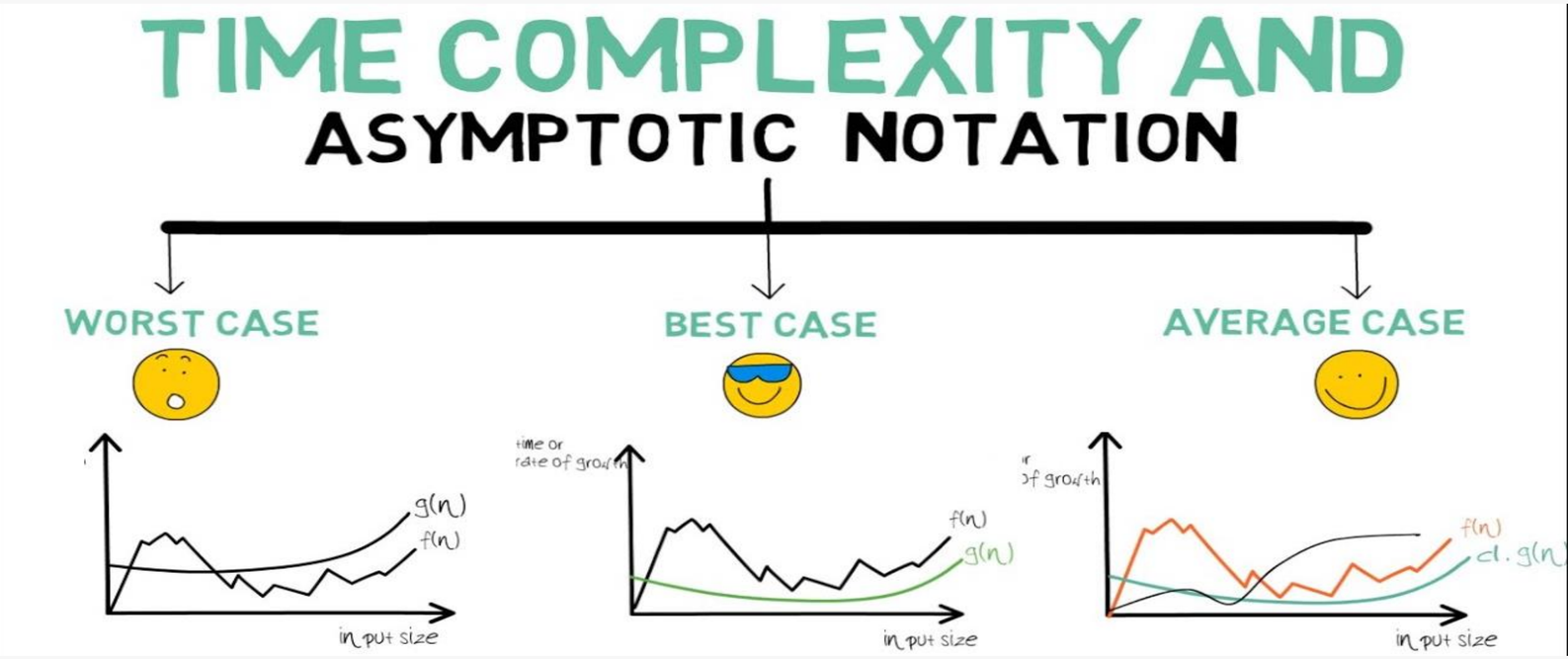
Através da identificação das operações primitivas e da quantidade de vezes que são executadas no algoritmo de identificação de elementos duplicados em um array chegamos à seguinte função

$$f(n) = 3 * c + 2 * c * n + 3 * n^2/2 + c * (n^2 + n)/2$$

Aplicando as diretrizes de simplificação, temos que $f(n) = \Theta(n^2)$. Isso quer dizer que $f(n)$ tem a mesma ordem de crescimento que uma quadrática

Análise Assintótica

Quando observamos tamanhos de entrada grande o suficiente para tornar relevante apenas a ordem de crescimento do tempo de execução, estamos estudando a **eficiência assintótica**



- Há pouco, aplicando as diretrizes de simplificação, aprendemos que funções complexas podem ser mapeadas para classes de funções sobre as quais conhecemos o crescimento $(n, \log n, n^2, etc)$
- Para ilustrar esse mapeamento utilizamos a notação Θ. Chegou a hora de entendermos o que essa notação significa

2

Notação Θ

Notação $\theta(n)$: Theta

Agora vamos definir formalmente o que significa essa notação. Para duas funções $f(n)$ e $g(n)$, dizemos que $f(n)$ é $\Theta(g(n))$ se

$$0 \leq c1 * g(n) \leq f(n) \leq c2 * g(n), \forall n \geq n_0$$

Em um resumo bem simplista ela está dizendo que se a gente "imprensar" $f(n)$ com $g(n)$ multiplicada por duas constantes diferentes, dizemos que $f(n)$ é $\Theta(g(n))$

Notação $\theta(n)$: Theta

Vamos ao exemplo. Lembra da função que descreve o tempo de execução da busca linear? Vamos tentar demonstrar que essa função é $\Theta(n)$

$$f(n) = 3 * c * n + 3 * c$$

O primeiro passo que vamos fazer é trocar as constantes por 1. Isso já foi dito antes. Usar c ou 1 tem o mesmo efeito. Assim, temos:

$$f(n) = 3n + 3$$

Notação $\theta(n)$: Theta

Agora vamos voltar a inequação. Como "desconfiamos" que $f(n) = 3n + 3$ é $\Theta(n)$, escolhemos $g(n) = n$

$$0 \leq c1 * n \leq 3n + 3 \leq c2 * n, \forall n \geq n_0$$

Agora precisamos encontrar valores para $c1$ e $c2$ para que essa inequação seja verdadeira. Vamos tentar com c1=1 e c2=6.

$$0 \leq n \leq 3n + 3 \leq 6 * n, \forall n >= n0$$

Se verificarmos com $n = 1$, vemos que a inequação é verdadeira:

$$0 \leq 1 \leq 6 \leq 6$$

Não é difícil também notar que $\forall n > 1$ ela sempre será verdadeira. Conseguimos, então, demonstrar que $f(n) \in \Theta(n)$, pois $g(n) = n$, limita inferior e superiormente $f(n)$

2

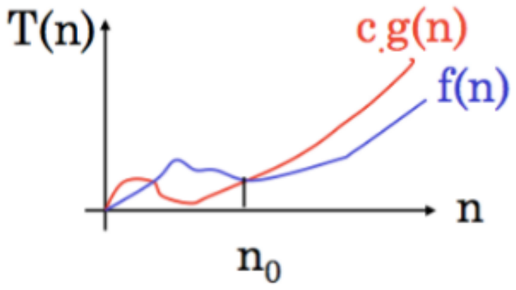
Notação O (Big O)

Notação O (Big O)

Enquanto a notação Θ define os limites inferior e superior de uma função, a notação O define apenas o limite superior. Ou seja, define um teto para uma determinada função

Para duas funções $f(n)$ e $g(n)$, dizemos que $f(n)$ é $O(g(n))$ se:

$$0 \leq f(n) \leq c * g(n), \forall n \geq n_0$$



Notação O (Big O)

Suponha que a função $n^2 + 1$ descreva o custo de execução de um algoritmo. Se aplicarmos as abstrações simplificadoras, desconfiamos que $f(n) \in O(n^2)$, certo?

Vamos demonstrar formalmente

$$0 \leq n^2 + 1 \leq c * n^2, \forall n \geq n_0$$

Se escolhermos $c = 2$ e $n_0 = 1$, temos:

$$0 \leq 2 \leq 2$$

Notação O (Big O)

Por exemplo

a função $f(n) = 7n$ é limitada superiormente por n^2 e, portanto, é $O(n^2)$. Todavia, não podemos dizer que ela é $\Theta(n^2)$ porque não há constante multiplicadora que, para n suficientemente grande, faça com que $c_1 * n^2$ seja menor do que $7n$

Por exemplo

Basta escolhermos uma função com n elevado a um expoente maior do que o da função sob análise que conseguimos definir um limite superior para ele.

$f(n) = n^2 \in O(n^2), \quad O(n^3), O(n^4)$ e assim por diante

2

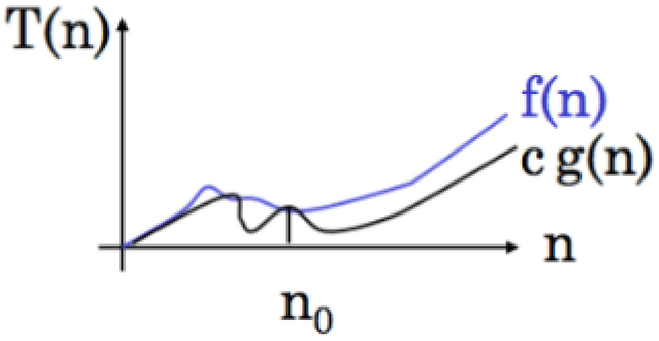
Notação Omega (Ω)

Notação Ω (Omega)

A notação Θ define o limite inferior e superior. O define apenas o limite superior. E Ω? Acertou. Apenas o limite inferior. Para duas funções $f(n)$ e $g(n)$, dizemos que $f(n)$ é $\Omega(g(n))$ se:

$$0 \leq c * g(n) \leq f(n), \forall n \geq n_0$$

A figura abaixo ilustra essa relação.



Notação Ω (Omega)

O processo para demonstrar que $f(n)$ é $\Omega(g(n))$ é muito semelhante, mas nesse caso precisamos achar apenas os valores de c e n_0

$$0 \leq c * n^2 \leq n^2 + 1, \forall n \geq n_0$$

Se escolhermos $c = 1$ e $n_0 = 1$, temos:

$$1 \leq 2$$

Exemplo. Suponha que a função $n^2 + 1$ descreva o custo de execução de um algoritmo. Se aplicarmos as abstrações simplificadoras, desconfiamos que $f(n) \in \Omega(n^2)$, certo? Vamos demonstrar formalmente.

Notação Ω (Omega)

Explicação

É simples definir um limite inferior para qualquer função. Basta utilizar o expoente 0. Ou seja, todas as funções são $\Omega(1)$. Mais do que isso, podemos escolher expoentes menores

Por exemplo

a função $f(n) = n^2$ é $\Omega(n), \Omega(\log n)$ e $\Omega(1)$. Todavia, faz mais sentido escolhermos uma função com o mesmo expoente, porque a informação é mais precisa. Ou seja, se uma função é quadrática, dizemos que ela é $\Omega(n^2)$

Resumo

Análise Assintótica	Análise assintótica leva em consideração grandes entradas para tornar relevante apenas a ordem de crescimento das funções de tempo de execução.
Ignore	Na prática, ignoramos as constantes e os expoentes de menor magnitude.
Simplificação	Usamos análise assintótica para simplificar a comparação entre funções
Notações	Usamos notações para descrever as classes de complexidade das funções.
Símbolos	▪ $O \rightarrow \leq$ ▪ $\Omega \rightarrow \geq$ ▪ $\Theta \rightarrow =$

Dúvidas, sugestões ou reclamações?

...

Alguma pergunta?

UEPB
Universidade
Estadual da Paraíba

Referências

1. BRUNET, João Arthur. **Análise Assintótica**. 2019. UFCG.
2. JÚNIOR, Fábio Luiz Leite. **Estrutura de dados e algoritmos**. 2012. UEPB.
3. CORMEN, Thomas H. **Algoritmos: teoria e prática**. 3ª edição. Rio de Janeiro : Elsevier, 2012.
4. LEE, Orlando. **Estrutura de dados**. 2022. Unicamp.
5.

UEPB
Universidade
Estadual da Paraíba